MILK Dependency Reduction

Comprehensive Refactoring Overview

milk-org/milk

March 2026

This document summarizes a systematic effort to reduce coupling and improve modularity across the MILK framework. 7 refactoring phases were evaluated: 2 were newly implemented, 3 were verified as already complete, 1 was determined impractical, and 1 was deferred.

1. Motivation

The MILK framework provides a real-time image processing pipeline for adaptive optics (AO) systems. Over time, several coupling points had grown between layers:

- COREMOD libraries mixed computation with CLI registration, forcing standalone executables to link the full CLIcore dependency chain
- FPS utilities linked CLIcore despite only using the lightweight FPS library functions
- Core code depended on GSL for random number generation even though GSL was only needed for specialized math
- Modules accessed the internal data.image[] array directly instead of using accessor functions
- The CLIcore.h mega-header (439 lines) was included everywhere even when only a small fraction was needed

The goal was to systematically evaluate each coupling point, implement fixes where practical, confirm prior work, and document the final state.

2. Phase Overview

Phase	Description	Status	Action
A (#5)	COREMOD compute/CLI split	DONE	Implemented
B (#1+#2)	FPS tools CLIcore removal	DONE	Verified
C (#3)	streamCTRL decoupling	N/A	Impractical
D (#6)	GSL removal from core	DONE	Verified
E (#4)	data.image[] accessor split	DONE	Verified
F (#7)	CLIcore.h targeted includes	DONE	Implemented
G (#8)	COREMOD/ISIO decoupling	DEFERRED	Low priority

3. Phase A: COREMOD Compute/CLI Split

IMPLEMENTED

Problem

The four COREMOD libraries (tools, iofits, arith, memory) each compiled computation functions and CLI registration wrappers into a single shared library. Standalone executables linked the full libraries, pulling in CLI argument types (CMDARGTOKEN, CMD, DATA), the module init constructors, and the entire CLIcore dependency chain -- none needed at runtime.

Approach

A compile-time separation using #ifndef MILK_NO_CLI guards was applied to ~90 COREMOD source files. CLI registration functions (_addCLIcmd, init_module_CLI, CLI wrappers) were guarded; pure computation was left unconditional. New CMake _compute library targets compile the same sources with -DMILK_NO_CLI:

```
  milkCOREMODtools       -> milkCOREMODtools_compute
milkCOREMODiofits      -> milkCOREMODiofits_compute
```

```
milkCOREMODarith       -> milkCOREMODarith_compute
milkCOREMODmemory      -> milkCOREMODmemory_compute
```

The add_milk_standalone() and add_cacao_standalone() CMake helper functions were updated to link _compute variants. All 92 standalone executables now link only computation code.

Results

Module	Full	Compute	Savings
COREMOD_tools	37 KB	36 KB	3%
COREMOD_iofits	81 KB	58 KB	29%
COREMOD_arith	446 KB	313 KB	29%
COREMOD_memory	365 KB	228 KB	37%
Total	930 KB	635 KB	31%

All four _compute libraries are completely clean of CLI symbols (verified via nm). All 92 standalone executables compile, link, and pass runtime tests.

4. Phase B: FPS Tools CLIcore Removal

VERIFIED COMPLETE

Rationale

The FPS (Function Parameter Structure) utilities are critical operational tools used to configure and control running processes. Tools like milk-fps-set, milk-fps-confstart, and milk-fps-runstart are called by deployment scripts (cacao-fps-deploy-v2) and operational workflows. If these tools could run without CLIcore, they would be available in USE_CLI=OFF builds -- essential for embedded or restricted AO deployments.

Prior Implementation

The FPS library was architecturally split into multiple layers during a prior refactoring effort:

- milkfps: Core FPS operations (create, connect, read/write parameters, save/load). No CLI dependency.
- milkfpsCLI: CLI registration wrappers that expose FPS functions as milk CLI commands. Depends on CLIcore.
- milkfpsStandalone: Standalone data providers (fps_standalone_data.c) for global symbols like 'data' and 'CLIPID' that would otherwise require CLIcore.
- milkfpsTUI: TUI display (fpsCTRL screen). Depends on ncurses and CLIcore.

Verification

We confirmed that all 10 FPS utility executables link only milkfps -- not CLIcore or milkfpsCLI:

Executable	Links To	CLIcore?
milk-fps-set	milkfps	No
milk-fps-list	milkfps	No
milk-fps-search	milkfps	No
milk-fps-info	milkfps	No
milk-fps-rm	milkfps	No
milk-fps-confstart	milkfps	No
milk-fps-confstop	milkfps	No
milk-fps-runstart	milkfps	No
milk-fps-runstop	milkfps	No
milk-fps-confstep	milkfps	No

Status: No changes needed. All 10 utilities are already standalone-capable, available in USE_CLI=OFF builds.

5. Phase C: streamCTRL CLIcore Decoupling

NOT PRACTICAL

Investigation

The goal was to extract the 7 streamCTRL TUI source files from libCLIcore.so into a standalone library, allowing milk-streamCTRL to run without CLIcore. We performed a line-by-line dependency analysis of all 7 files.

Findings

The TUI code is architecturally coupled to CLIcore's internal infrastructure:

- Debug macros: PRINT_ERROR, DEBUG_TRACEPOINT, WRITE_FULLFILENAME, EXECUTE_SYSTEM_COMMAND (from milkDebugTools.h, part of CLIcore)
- Signal handling: set_signal_catch() -- CLIcore's signal handler (standalone stub insufficient for full TUI behavior)
- TUI framework: TUI_printfw, TUI_newline, TUI_exit, TUI_init_terminal, TUI_clearscreen, screenprint_setreverse/color/bold -- all provided by milkTUI, part of CLIcore's TUI subsystem
- Global state: CLIPID (process ID), data.processname (process naming), SHAREDSHMDIR for stderr redirection

Extracting these files would require duplicating or re-exporting the debug macro system, TUI framework, and signal infrastructure in a parallel standalone library. The cost-benefit ratio is unfavorable -- streamCTRL is a monitoring TUI that inherently needs the TUI infrastructure it runs within.

Status: Determined impractical. No changes made. The dependency is structural and appropriate.

6. Phase D: GSL Removal from Core

VERIFIED COMPLETE

Rationale

GNU Scientific Library (GSL) is a heavyweight dependency that pulls in CBLAS, error handling, and many math routines. Its presence in core MILK forced every standalone executable to link GSL -- adding several MB to the dependency chain. The goal was to eliminate GSL from core MILK, restricting it to optional plugins that genuinely need it.

Prior Implementation

GSL was replaced in core MILK through a series of targeted changes:

- Native RNG: A pure-C random number generator (xorshift64* algorithm) was implemented in milkdata.h/c as MILK_RNG. Functions milk_rng_init(), milk_rng_uniform(), milk_rng_gaussian(), and milk_rng_poisson() replace gsl_rng_* calls.
- CLIcore unlinked: libCLIcore.so no longer links GSL_LIBRARIES or FFTW_LIBRARIES. These libraries are managed by the plugins that use them (e.g., linopt_imtools for GSL SVD, fft for FFTW).
- Build system: pkg_check_modules(GSL gsl) was moved to the top-level CMakeLists.txt so that plugins can still discover GSL independently.
- Header cleanup: fftw3.h is no longer included by core headers -- FFT operations are strictly limited to the fft plugin.

Verification

Confirmed that no COREMOD file or core library references GSL functions. The only GSL usage is in optional plugin modules (linalgebra for eigenvalue decomposition).

```
  # xorshift64* RNG in milkdata.h/c
void milk_rng_init(uint64_t seed);
double milk_rng_uniform(void);
double milk_rng_gaussian(double sigma);
long   milk_rng_poisson(double lambda);
```

Status: No changes needed. GSL is plugin-only. Core MILK uses the native MILK_RNG implementation.

7. Phase E: data.image[] Accessor Split

VERIFIED COMPLETE

Rationale

The monolithic DATA struct (defined via CLIcore.h) contains data.image[], data.variable[], and other internal arrays. Direct access like data.image[id] couples every module to CLIcore's internal memory layout. If modules instead used accessor functions (image_ID(), create_image_ID(), etc.), they would be insulated from layout changes and could compile without the full DATA struct definition.

Prior Implementation

This migration had been completed systematically across all COREMOD modules:

- Accessor functions in COREMOD_memory: image_ID() returns the imageID for a named image, create_image_ID() creates a new image and returns its ID, read_sharedmem_image_ID() maps a shared memory image.
- All COREMOD files now use the imageID return value and the IMGID/dcimg/dcnimg accessor macros to work with image data, rather than indexing data.image[] directly.
- The IMGID type and dcimg/dcnimg accessor macros were introduced in milkdata.h to provide a clean, type-safe interface for referencing images by ID.

Verification

Grep across all 110 COREMOD source files confirms zero direct data.image[] accesses:

```
  $ grep -rn 'data\.image\[' src/COREMOD_*/*.c
(no matches -- 0 direct accesses)
```

Status: No changes needed. All image access uses accessor functions and IMGID macros.

8. Phase F: CLIcore.h Targeted Includes

IMPLEMENTED

Problem

CLIcore.h is a 439-line mega-header that transitively includes std headers, milkdata.h, ImageStreamIO.h, fps.h, processtools.h, milkDebugTools.h, CLI types (CMD, CMDARGTOKEN, DATA), module management macros, and TUI stubs. Most compute-only source files only need a small subset.

Approach

We categorized all 110 COREMOD source files by their actual CLIcore.h dependency. 25 compute-only files were identified that use no CLI registration. For these, CLIcore.h was replaced with targeted sub-headers:

```
  #ifdef MILK_NO_CLI
#include "CLIcore_standalone.h"
#else
#include "libmilkdata/milkdata.h"  // types
#include "milkDebugTools.h"        // debug macros
#include <fps.h>                   // FPS if needed
#endif
```

Results

Metric	Before	After
COREMOD files with CLIcore.h	100	75
Files with targeted includes	0	25
Build status	Pass	Pass
Standalone executables	All pass	All pass

The 75 remaining files legitimately need CLIcore.h for CLI registration (RegisterCLIcommand, CMDARGTOKEN, INIT_MODULE_LIB).

9. Phase G: COREMOD/ImageStreamIO Decoupling

DEFERRED

ImageStreamIO provides the fundamental IMAGE struct type that all computation operates on. It is lightweight, always needed at runtime, and its API is stable. Abstracting it away would add complexity without practical benefit.

Status: Deferred. Not worth pursuing.

10. Gains in Maintainability and Modularity

Clear Separation of Concerns

Each COREMOD now has two distinct layers: a compute layer (_compute) containing pure functions with no CLI dependencies, and a full CLI layer with thin wrappers. FPS utilities operate independently of CLIcore. GSL is confined to plugins. Image access goes through typed accessors rather than raw array indexing.

Safer Refactoring

Changes to CLIcore internals (command registration, argument parsing, module loading) no longer risk breaking compute code. The #ifndef MILK_NO_CLI guards act as a firewall. Similarly, GSL version changes or removal cannot affect core MILK functionality.

Explicit Dependencies

Targeted includes make each file's dependencies self-documenting. milkdata.h means image types; milkDebugTools.h means debug macros; fps.h means FPS framework. The IMGID/dcimg/dcnimg accessor pattern makes image array access explicit and safe.

Standalone Deployment Readiness

The combined effect of all phases means MILK can be built with USE_CLI=OFF to produce a minimal deployment containing only standalone executables, FPS utilities, and the core computation libraries. No readline, ncurses, bison, flex, GSL, or FFTW required. This is critical for embedded AO systems and restricted deployment environments.

Faster Incremental Builds

25 files no longer parse the 439-line CLIcore.h and its transitive includes. Compute-only libraries can be rebuilt without recompiling CLI wrappers.

Foundation for Extension

The _compute library pattern and MILK_NO_CLI / CLIcore_standalone.h mechanism are reusable. Plugin modules can adopt the same pattern. The milkfps 4-layer split (core / CLI / standalone / TUI) serves as a proven reference architecture.

11. Complete Summary

New Code Changes (this session)

- Phase A: ~90 COREMOD source files guarded with #ifndef MILK_NO_CLI. 4 new _compute library targets created. Standalone helpers updated.
- Phase F: 25 COREMOD source files switched from CLIcore.h to targeted sub-headers.

Verified Prior Work

- Phase B: 10 FPS utility executables confirmed standalone-capable (link milkfps only, not CLIcore). milkfps

library architecture split into core/CLI/standalone/TUI layers.

- Phase D: GSL eliminated from core MILK. Replaced with native xorshift64* MILK_RNG in milkdata.h/c. libCLIcore.so no longer links GSL or FFTW. GSL confined to optional plugins.
- Phase E: Zero direct data.image[] accesses in COREMOD. All image access uses IMGID type and accessor functions (image_ID, create_image_ID). Modules insulated from DATA struct layout changes.

Analysis Results

- Phase C: streamCTRL TUI deeply coupled to CLIcore debug, signal, and TUI infrastructure. Extraction would require duplicating CLIcore internals.
- Phase G: ImageStreamIO is lightweight and fundamental. Decoupling not worthwhile.

Key Invariants Maintained

- Full CLI build (USE_CLI=ON) produces identical libraries with no behavioral changes
- All 92 standalone executables compile, link, and pass runtime tests (-h1)
- No API changes -- all function signatures preserved